



INSTITUTO DE FORMACIÓN TÉCNICA SUPERIOR N.º 18

Tecnicatura Superior en Ciencia de Datos e Inteligencia Artificial



```
> git push origin main  
$ demo.launch(server_name="0.0.0.0")  
> Deploying to Render...  
> Live ✓
```

SEMINARIO DE ACTUALIZACIÓN

Clase 5: Del link que se cae al deploy propio

Dónde vive tu aplicación · Hosting, PaaS y Render · Armá tu portfolio

DOCENTE

Cynthia M. Villagra

CICLO LECTIVO

2026 · Segundo cuatrimestre

FECHA

Viernes 18 de septiembre de 2026

Qué vamos a hacer hoy

1 Por qué se cayó tu link

El de share=True, el que mandaste y dejó de andar

2 Dónde puede vivir una app

Hosting, stacks, VPS, PaaS y las nubes

3 WSGI y ASGI

Por qué no cualquier plataforma sirve para Gradio

4 Repaso

Cuestionario corto, sin nota

5 Deploy en Render

Tu app de Gradio, publicada

6 Lo mismo con Streamlit

El criterio se generaliza

7 Armá tu portfolio

Ticket de salida — la última hora de la clase

La clase pasada publicaste tu app

Corriste tu app con share=True y mandaste el link.



```
demo.launch(share=True)
```

```
> Running on public URL: https://a1b2c3d4.gradio.live
```

Alguien lo abrió y andaba. Después dejó de funcionar. ¿Por qué?

Antes de la respuesta: ¿qué creen que pasó?

Tirá tu hipótesis por el chat o por micrófono. No hay respuesta incorrecta todavía.

¿Expiró?

¿Los links tienen fecha de vencimiento?

¿Se rompió algo?

¿Falló el código, o la plataforma?

¿Dependía de algo?

¿De qué tenía que estar prendido para andar?

Si decís “expiró”: ¿y si lo probabas ese mismo día, con la compu apagada?

El link no apuntaba a ningún servidor

Apuntaba a tu computadora.

Lo que creías

Que tu app estaba “subida a internet”, como una página web cualquiera, y vivía en algún lado.

Lo que pasaba

Gradio abría un túnel desde tu máquina y le prestaba una dirección pública. Tu código seguía corriendo en tu compu. Cuando se cerró, se terminó.

Aunque el link durara un año, se caía igual al apagar la computadora.

¿Alguno contrató un hosting alguna vez?

Dominio, espacio en disco, casilla de mail, base de datos MySQL, certificado SSL.

Servidor

La computadora que corre tu app y atiende pedidos. Siempre prendida, siempre conectada.

Hosting

El servicio de alquilar esa computadora. Lo que cambia entre tipos es quién se ocupa de qué.

Ese hosting funciona perfecto. Para lo que está diseñado.

Un stack es una pila de capas

Lenguajes, frameworks, bases y herramientas que se combinan. Cada capa hace una cosa.

Frontend

lo que el usuario ve y toca

HTML · CSS · JavaScript

Backend

la lógica: procesa, decide, conecta

Python · Node.js · PHP

Base de datos

dónde se guarda la información

MySQL · PostgreSQL · MongoDB

Infraestructura

dónde corre todo para estar en internet

Linux · Render · AWS

Las tres primeras ya las trabajaron. Hoy la clase es sobre la cuarta.

OJO

Ese hosting está armado para otro stack

Viene con Apache, MySQL y PHP porque está hecho para LAMP. Python queda afuera por diseño.

Cómo ejecuta PHP

Cada visita levanta un script, lo ejecuta y lo mata. No queda nada corriendo entre un pedido y el siguiente.

Qué necesita Gradio

Un proceso que quede vivo, sosteniendo la cola y la conexión abierta con el navegador.

El mail y la base de ese hosting sí te sirven. El que no sirve es el de aplicación.

Los tipos, y quién se ocupa de qué

TIPO	VOS PONÉS	¿SIRVE PARA GRADIO?
Compartido	archivos	no — no deja procesos vivos
VPS	el sistema operativo entero	sí, pero lo mantenés vos
PaaS	el repositorio	sí — acá está el free tier
Serverless	una función suelta	no — no hay proceso vivo
Nube elástica	depende del servicio	sí, pero pide tarjeta

Para publicar Python gratis y sin tarjeta, lo que buscás es una PaaS.

AWS, Azure y Google Cloud no son un tipo

Son proveedores que venden todos los tipos. Por eso confunden.

Lo que querés	En AWS	En Google Cloud
VPS	EC2	Compute Engine
PaaS	App Runner	App Engine
Serverless	Lambda	Cloud Functions

“Elástico” = escala con la demanda. Sin límite configurado, la factura también.

No cualquier PaaS sirve: WSGI o ASGI

WSGI — el modelo viejo

Sincrónico: un pedido a la vez. Llega, se responde, se cierra.

Flask clásico, Django tradicional.

ASGI — asincrónico

Varios pedidos a la vez, y soporta WebSockets: una conexión que queda abierta en los dos sentidos.

Es lo que Gradio necesita.

Gradio usa WebSockets para su cola. Por eso Render funciona y Vercel no.

Mono-proceso o separado en capas

Gradio: mono-proceso

El mismo backend de Python sirve también el frontend ya armado. Una sola cosa que desplegar.

App en capas

El frontend vive separado del backend y le habla por API. Ejemplo: un frontend en Vercel que consulta una API en otro lado.

Vercel no te sirve para publicar Gradio, pero sí como ejemplo de esta separación.

Repaso rápido, sin nota

Respondé por el formulario o por el chat, como te sea más cómodo.

- ¿Por qué se cayó el link de share=True?
- ¿Qué capa del stack estamos resolviendo hoy?
- ¿Por qué una plataforma serverless no sirve para Gradio?
- Si una plataforma solo soporta WSGI, ¿qué problema vas a tener?

Si esta última no sale, volvemos dos diapositivas antes de seguir.

Render: lo miramos juntos

Lo hago yo y ustedes miran. Completarlo es la actividad de la semana — no se espera que todos terminen ahora.

Sin tarjeta

Plan gratuito real, no pide medio de pago.

Soporta ASGI

WebSockets incluidos:
Gradio anda.

Desde GitHub

Conectás el repo y cada push redploya.

Se duerme

A los 15' sin tráfico. Tarda ~1' en despertar.

Cuatro pasos

1

Crear la cuenta

En render.com, sin tarjeta de crédito.

2

Conectar el repositorio

New → Web Service → tu repo de GitHub.

3

Ajustar el código

Una línea. La de la diapositiva siguiente.

4

Deploy y esperar

Mirá el log. La primera vez tarda.

OJO

La línea que cambia todo

¿Qué error esperarás si dejás `server_name` en `localhost`? Votá antes de correrlo.

```
import os

demo.launch(
    server_name="0.0.0.0",
    server_port=int(os.environ.get("PORT", 7860))
)
```

localhost le dice a tu app que solo atienda pedidos de sí misma.

Tu app, viva sin tu computadora

Cerrá la notebook y probá el link. Ahora sí sobrevive.

Antes: share=True

Un túnel hacia tu máquina. Si apagabas la compu, se caía.

Ahora: Render

Un servidor que corre tu app. No depende de vos para seguir en pie.

Ojo: el plan gratuito duerme a los 15 minutos. El primer visitante espera ~1 minuto.

Lo mismo, con Streamlit

Otra herramienta para armar la interfaz. Mismo problema de infraestructura.

Qué cambia

El código de la interfaz. Otra librería, otra forma de escribir los componentes.

Qué no cambia

Necesita proceso persistente. Es PaaS. Hay que decirle en qué puerto escuchar. El deploy es el mismo.

No aprendiste a deployar Gradio. Aprendiste a deployar Python.

Ticket de salida: armá tu portfolio

Con lo que vimos hasta acá, dejá tu repositorio funcionando como portfolio: alguien que entra por primera vez tiene que poder entender qué hiciste en cada clase y reproducirlo.

1

Repositorio

Usá o renombrá el que mandaste en la planilla.

2

Una carpeta por clase

Desde Clase_02. La 01 no tuvo entregable técnico.

3

Adentro de cada una

Todo lo necesario para que otro pueda replicarlo.

4

Los README

Uno por carpeta, y uno general en la raíz como portada.

5

Entregá

git push y pegá el link del repositorio.

**Arrancaste con un link que se te cayó.
Terminás con una app publicada y un repo que la explica.**

HOY, ANTES DE IRTE

El link a tu repositorio de GitHub.
No lleva nota.

DURANTE LA SEMANA

El deploy en Render andando y el equivalente en Streamlit,
documentados en Clase_05.

LA PRÓXIMA

Van a abrir el repositorio de un compañero, sin que esté para explicarles nada.
Todo lo que hoy te pareció obvio, mañana lo va a leer otro.